

FPGA IMPLEMENTATION OF PRESENT ALGORITHM WITH IMPROVED SECURITY

Vallish Kumar Reddy^[1], Rao Surya^[2], Akhil Reddy^[3], P. Sathish Kumar^[4]

Department of Electronics & Communication Engineering

Amrita School of Engineering, Bengaluru

Amrita Vishwa Vidyapeetham

India

vallishkumarreddy@gmail.com^[1], rsurya1198@gmail.com^[2], akhilreddy199706@gmail.com^[3], p_sathishkumar@blr.amrita.edu^[4]

Abstract—In recent times, the need of information security has grown widely and especially the study of lightweight symmetric ciphers among all the cryptographic algorithms has gained huge interest. The reason for this enlarging demand for cryptography is due to the ever-increasing insecurities particularly in the resource limited devices like the IoT (Internet of Things). The prime focus of this paper is to consider the variety of hardware architectures of a lightweight cipher PRESENT, which acquired its fame as a resource conservator as well as a tough cipher. This lightweight cipher can be used in Home automation and IoT related devices. In this paper, individual focus is laid on area minimization and security improvement. The 4-bit iterative architecture implemented in this paper saves area which makes it suitable for resource constraint devices like RFID Tags. Also, a modified key-dependent architecture is implemented with high potential to resist security threats which can be used in IoT setups where data suffers severe breaches.

Keywords: FPGA; security; cryptography; lightweight; PRESENT

I. INTRODUCTION

IoT can be viewed as a perfect application of Internet in this smart planet with huge amount of electronics in every domain and its importance is growing with the growth in the use of smart objects [1]. But there is also a possibility of threat with this lavish use of IoT. The reason for this threat is the ever-escalating insecurities in data transfer [2]. Thus, the security and privacy of these smart devices should be taken care. However, it is not very easy to make the IoT appliances secure and private, especially in this fast-growing world of technologies [3]. Today a large amount of sensitive information in defense, financial, medical world, and others are more vulnerable to physical attacks as the device computing lack potential in security. Taking these hindrances into consideration IoT appliances should be made more secure and private [17].

The IoT setup consists of a lot of web-connected smart objects, mini embedded controllers and processors (generally of low end [6]), hardware required for sensing different kinds of data and for communicating with other devices, along with a network [5]. These devices also communicate with each other and based on the mutual information they got and

perform their tasks without any human involvement. Thus, a security solution to the problems mentioned above is achieved only through hardware implementations of cryptographic models [4].

The use of FPGAs plays a major role for the development of hardware-based solutions. The operations that are usually used in cryptographic algorithms are realized easily using FPGAs [8]. The FPGA implementations are becoming popular because of its advantages in the design time, performance, power consumption, chip area. Moreover, these days the protocols are always advancing and they need to be replaced by the new ones. This is a perfect reason which necessitate the use of FPGAs with their unique potential to get reprogrammed when required [9]. Also, the Symmetric cryptographic techniques are suitable in IoT ecosystems as they are more resource-friendly when compared to the rest [7]. In 2012 PRESENT, a lightweight cipher, is standardized (by International Electrotechnical Commission) as the symmetric block-cipher designed to the constrained systems [11].

A. Lightweight cryptography

Since the invention of first cipher, cryptographic algorithms have been advancing till date. Lot of cryptographic concepts have been improvising according to the trends in technology. In this continuous progress Lightweight cryptography (LWC) has made its mark [10]. Lightweight cryptography is a sub-branch of cryptography which is designed for resource-constrained systems like IoT based sensors, RFID applications, smart devices in medical field and so on. LWC is a perfect match to the resource limited platforms which saves area, power and reduces cost.

B. Why is lightweight cryptography required for IoT?

The IoT is a boon to this present world only if it has proper security [15]. Though the conventional cryptography provides security to IoT, it is not so practical because of the cost needed to embed security with the device. So, there is no alternative except to make cryptography suitable to these

circumstances (less area and less cost), which led to the growth of Lightweight cryptography. With the increasing growth in IoT, Lightweight cryptography has a great scope in the coming days.

II. PRESENT

PRESENT comes under the category of symmetric ciphers, which germinated in orange labs in 2007. It is later standardized (in 2012) by International Standards Organization as “[ultra-lightweight block cipher] suitable for lightweight cryptography, which is designed for implementation in resource limited environments” [11]. It satisfies an important cryptographic property called as the Avalanche effect, thus ensuring the level of encryption.

The PRESENT algorithm works on Substitution-Permutation Network (SPN). SPN is a cryptographic concept which has a set of rules to be followed to ensure the level of encryption. SPN network has two major components namely substitution layer and permutation layer along basic mathematical operations like XOR and shifting or rotation of bits.

Generally, SPN has multiple rounds of encryption with each round involving both the substitution and permutation layers along with the key (which transforms in every round of encryption). It is very convenient to do these kind of encryption operations using simple hardware setup which makes SPN a very important component in lightweight ciphers, especially in PRESENT.

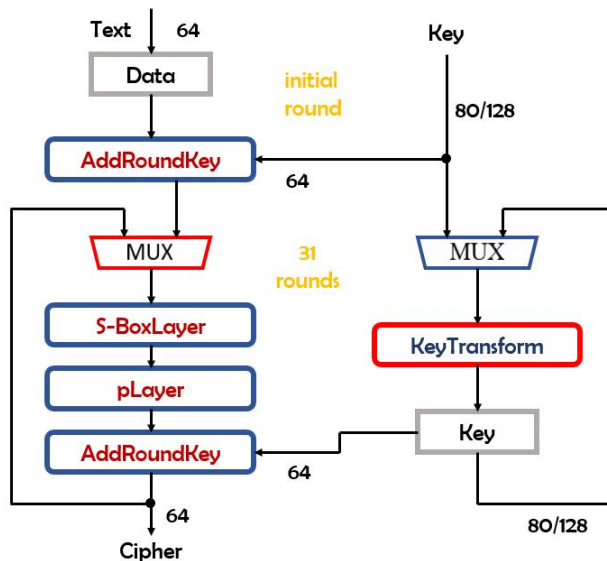


Fig. 1. Block diagram of PRESENT

The input data used for the PRESENT encryption is fixed as 64-bit block. It has two available sizes for key which are 80-bit & 128-bit. The size of the key (80/128-bit) is

chosen depending upon level of security needed for respective applications [16].

The PRESENT cipher uses three basic operations (as shown in Fig 1), which involves all the complexity to ensure the level of encryption of data: [4]

- 1) AddRoundKey: It involves XOR operation of 64-bits of data with 64 most significant bits of key.
- 2) S-boxLayer: It is a 4-bit operation which replaces the input bits with the respective output bits.
- 3) P-boxLayer: It involves mapping of individual bits of data according to the standards.

A. S-box

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S[x]$	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

S-box is a component in the encryption process which substitutes 4 input bits with respective output bits, based on the standard. It gives the output bits to the P-box followed. Since data in PRESENT encryption is 64-bit, 16 S-boxes are required for the substitution layer.

B. P-box

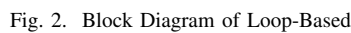
i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	16	32	48	1	17	33	49	2	18	34	50	3	19	35	51
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P(i)$	4	20	36	52	5	21	37	53	6	22	38	54	7	23	39	55
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
$P(i)$	8	24	40	56	9	25	41	57	10	26	42	58	11	27	43	59
i	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
$P(i)$	12	28	44	60	13	29	45	61	14	30	46	62	15	31	47	63

A P-box is immediately followed by an S-box and it does the permutation of bits (rearranging the bits of data). These rearranged or mixed output bits follow the further encryption process. The standard PRESENT P-box ensures that it mixes the bits well enough so that the algorithm satisfies the Avalanche criteria.

C. Key Generation

In each round the key is generated as:

- 1) 61-bit rotation of key (bitwise).
- 2) The 4 (key size = 80-bit) or 8 (key size = 128-bit) most significant bits are passed to S-box.
- 3) 5 intermediate bits are XORed with a 5-bit counter, which gets updated in each round.

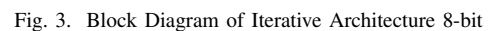


A. Loop-Based Architecture [13]

To begin with, the multiplexers hold in load state where data gets loaded. Then the multiplexers switch to encryption state where the 31 encryption rounds get performed. This typical Loop-Based architecture even though has a good latency, it suffers a few disadvantages with a lot of scope of improvement in area.

This architecture has a different approach to load the data. Instead of loading data in one clock cycle (like in the traditional Loop-Based architecture), it loads in multiple cycles. Since it loads 8-bits at once, it takes totally 10 cycles to load data for an 80-bit key and 16 cycles for 128-bit key. However, the size of the registers used, and the rest of the encryption process remain same as in the Loop-Based architecture. But here the multiplexers used for switching between states will be of less size as the datapath is reduced

Difference: In the first architecture the load state and the encryption state are controlled by two multiplexers of 80-bit and 64-bit. But in this architecture instead an 8-bit 2-to-1 multiplexer is used which decreases the area by a considerable amount.



It is a unique concept proposed in [12] where the security of PRESENT cipher is improved using a variable S-box instead of fixed S-box. To apply this concept into the algorithm, it takes a few steps:

- ### 1) Filtering the best S-boxes

The concept mentioned in [12] uses 16 S-boxes. The S-box should follow the same characteristics of the PRESENT S-box and it should follow the characteristics of the cryptanalysis (both differential and linear) in order to be used in the algorithm. Cryptanalysis is a study of cryptosystems with the objective of attacking them and decrypting codes and ciphers. Cryptanalysis generally falls into one of the several categories of testing the algorithm which ensures the level of security of any component of encryption.

So, in order to perform linear and differential cryptanalysis (to a S-box) there are certain steps and equations to be followed. [12] explains the whole process of filtering 16 S-boxes using both the types of cryptanalysis i.e. differential & linear.

The 16 S-boxes chosen should satisfy both the types of cryptanalysis for sure. Those are:

The 16 S-boxes are:

S0=[3,8,15,1,10,6,5,11,14,13,4,2,7,0,9,12];
S1=[15,12,2,7,9,0,5,10,1,11,14,8,6,13,3,4];
S2=[8,6,7,9,3,12,10,15,13,1,14,4,0,11,5,2];
S3=[0,15,11,8,12,9,6,3,13,1,2,4,10,7,5,14];
S4=[1,15,8,3,12,0,11,6,2,5,4,10,9,14,7,13];
S5=[15,5,2,11,4,10,9,12,0,3,14,8,13,6,7,1];
S6=[7,2,12,5,8,4,6,11,14,9,1,15,13,3,10,0];
S7=[1,13,15,0,14,8,2,11,7,4,12,10,9,3,5,6];
S8=[0,3,5,8,6,9,12,7,13,10,14,4,1,15,11,2];
S9=[0,3,5,8,6,12,11,7,9,14,10,13,15,2,1,4];
S10=[0,3,5,8,6,10,15,4,14,13,9,2,1,7,12,11];
S11=[0,3,5,8,6,12,11,7,10,4,9,14,15,1,2,13];
S12=[7,12,14,9,2,1,5,15,11,6,13,0,4,8,10,3];
S13=[4,10,1,6,8,15,7,12,3,0,14,13,5,9,11,2];
S14=[2,15,12,1,5,6,10,13,14,8,3,4,0,11,9,7];
S15=[15,4,5,8,9,7,2,1,10,3,0,14,6,12,13,11];

2) Build a procedure for choosing an S-box

It is the part that is responsible for increasing the complexity of PRESENT algorithm by making the S-box dependent on key. Here in each round a variable is used to choose the S-box. That variable is calculated by XORing the key bits using reduction operator which generates a 4-bit value. This 4-bit variable represents numbers from 0 to 16 and is enough to choose one S-box out of the 16 available S-boxes.

SECURITY DISCUSSION

Both the sub-fields of cryptanalysis occupy a key place in building the strength of the algorithm. The PRESENT algorithm has lot of encrypting elements out of which the S-box is a very prominent block (usually referred as the heart of PRSESNT). Cryptanalysis ensures the strength of the S-box which in turn strengthens the PRESENT algorithm. In this improved concept S-box is chosen in every round by XOR operation between elements of key. Since the key is a random value, the choosing of S-box becomes random which increases the security of the algorithm. The concept and working of differential & linear cryptanalysis are better explained in [12], which gives a better idea of how it works on a S-box. So, for each S-box 16 values are there and finding the correct one generates multiple probabilities and can be mathematically viewed as $16 (2^4)$. That means the typical PRESENT cipher implementation has $2^4.16 = 2^4.2^4 = 2^8$ different chances. But the model in [12] has different chances of $2^8.16 = 2^8.2^4$, which is due to the variable S-boxes. So, it is concluded that the attackers find it more difficult to break the cipher and

understand the information for this updated PRESENT cipher than for the conventional PRESENT.

D. Proposed Architectures

1) Iterative Architecture (4-bit)

The 4-bit iterative architecture which is explained in this paper is like the architecture reported in [14]. Here the size of the multiplexers is reduced to 4-bit which is lesser than the existing architecture. This change slightly reduces the area of the architecture.

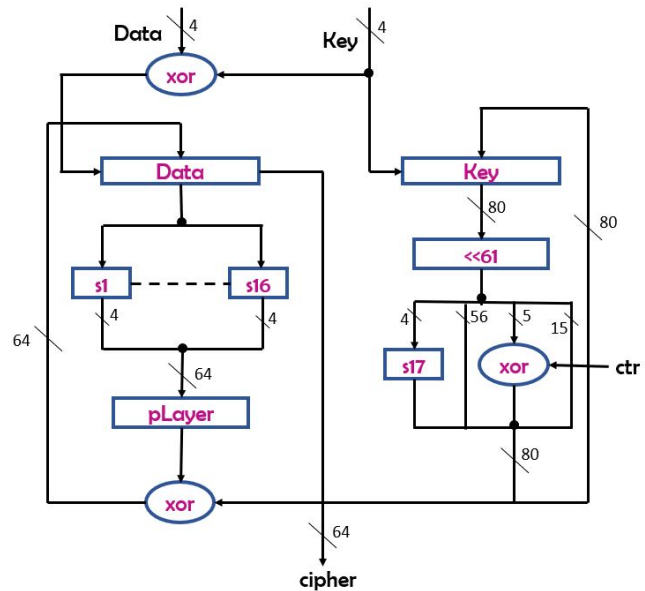


Fig. 4. Block Diagram of Iterative Architecture 4-bit

For the encryption process it takes 31 cycles, but for loading data it takes 20 (key size = 80-bit) or 32 (key size = 128-bit) cycles. This makes the latency as 51 or 63 cycles. The hardware of this architecture consists of 149 flipflops, 73 XOR gates, 17 S-boxes, a P-box and a bit rotator. But, the P-box and the bit rotator used for shifting does not require any hardware.

2) **Modified Key Dependent Architecture** This proposed architecture is inherited from the design explained in [12]. As mentioned in the section C, [12] explains the concept of making the S-box dependent on key but it lacks the implementation details of the algorithm. Also, the way of choosing S-box which is explained in [12] is almost impractical to implement, as it fails in decryption process of cipher text. However the proposed architecture uses a slightly different concept of choosing the S-boxes.

Calculation of value for choosing S-box In the proposed architecture the key used for calculation of value which chooses the S-box, is updated (shifted a few bits) irrespective of the typical key update process

or key transformation that happens in the encryption process.

This way of calculating the value which chooses the S-box, eases the decryption process, because the while decrypting, we know the number of times the key got shifted during the encryption process.

The proposed architecture uses the loop-based model explained in [13], with the inclusion of 16 S-boxes for greater security. The security of the implemented architecture is high as achieved and discussed in [12]. It is important to note that the implemented architecture only focuses in increasing the security which creates a great scope of improvement in terms of area.

IV. RESULTS

Here the results of different architectures are shown with input test vector:

plaintext = 0000000000003b1e

key = 00000000000050ba282

A. Simulation Result of the Loop-Based Architecture

Encryption:

Name	Value
outdata[63:0]	7bd5a67915ed6ca3
clk	1
load	0
indata[63:0]	0000000000003b1e
key[79:0]	00000000000050ba282
i[31:0]	00000041

ciphertext = 7bd5a67915ed6ca3

Decryption:

Name	Value
outdata[63:0]	0000000000003b1e
clk	1
load	0
indata[63:0]	7bd5a67915ed6ca3
key[79:0]	bb027a0eb8783af4f704
i[31:0]	00000041

plaintext = 0000000000003b1e

B. Simulation Result of Iterative Architecture (8-bit)

Encryption:

Name	Value
outdata[63:0]	7bd5a67915ed6ca3
clk	1
load	0
load2	1
indata[7:0]	00
key[7:0]	00
i[31:0]	00000053

ciphertext = 7bd5a67915ed6ca3

Decryption:

Name	Value
outdata[63:0]	0000000000003b1e
clk	1
load	0
load2	1
indata[7:0]	7b
key[7:0]	bb
i[31:0]	00000051

plaintext = 0000000000003b1e

C. Simulation Result of Iterative Architecture (4-bit)

Encryption:

Name	Value
outdata[63:0]	7bd5a67915ed6ca3
clk	1
load	0
load2	1
indata[3:0]	0
key[3:0]	0
i[31:0]	00000067

ciphertext = 7bd5a67915ed6ca3

Decryption:

Name	Value
outdata[63:0]	0000000000003b1e
clk	1
load	0
load2	1
indata[3:0]	7
key[3:0]	b
i[31:0]	00000065

plaintext = 0000000000003b1e

D. Simulation result of Modified Key Dependent Architecture

Encryption:

Name	Value
outdata[63:0]	ca713a6ba6583e66
clk	1
load	0
indata[63:0]	0000000000003b1e
key[79:0]	00000000000050ba282
i[31:0]	1

ciphertext = ca713a6ba6583e66

Decryption:

Name	Value
outdata[63:0]	0000000000003b1e
clk	1
load	0
indata[63:0]	ca713a6ba6583e66
key[79:0]	7e99454ce038e9d5cae9
i[31:0]	00000043

plaintext = 0000000000003b1e

TABLE 1

Resource Usage for the 3 architectures with FPGA version xc6slx25-3fgg484

Arch	Data	Key	FF	LUT	SLC	IOB	LAT
[13]	64	80	149	239	68	210	32
[13]	64	128	197	322	103	258	32
[14]	64	80	149	171	47	83	41
[14]	64	128	197	208	57	83	47
C	64	80	149	175	46	75	51
C	64	128	197	206	57	75	63

TABLE 2

Resource Usage for the 3 architectures with FPGA version
xc3s400-5fg456

Arch	Data	Key	FF	LUT	SLC	IOB	LAT
[13]	64	80	149	348	224	210	32
[13]	64	128	197	305	205	258	32
[14]	64	80	149	229	121	83	41
[14]	64	128	197	281	146	83	47
C	64	80	149	226	116	75	51
C	64	128	197	277	142	75	63

TABLE 3

Resource Usage for the 3 architectures with FPGA version
xc5v1x50t-3ff1136

Arch	Data	Key	FF	LUT	SLC	IOB	LAT
[13]	64	80	149	213	64	210	32
[13]	64	128	197	261	77	258	32
[14]	64	80	149	213	55	83	41
[14]	64	128	197	268	72	83	47
C	64	80	149	216	54	75	51
C	64	128	197	267	72	75	63

TABLE 4

Resource Usage for the 3 architectures with FPGA version
xc4v1x25-12ff668

Arch	Data	Key	FF	LUT	SLC	IOB	LAT
[13]	64	80	149	348	224	210	32
[13]	64	128	197	305	205	258	32
[14]	64	80	149	229	120	83	41
[14]	64	128	197	281	146	83	47
C	64	80	149	226	116	75	51
C	64	128	197	277	142	75	63

V. CONCLUSION

The paper dealt with the comparison of various hardware architectures of PRESENT in terms of area occupancy. Two of the existing architectures are implemented and one is proposed, whose area consumptions are compared and analysed. The 4-bit iterative architecture with 80/128-bit key proposed in this paper occupies lesser LUTs than the 8-bit iterative architecture with 80/128-bit key. The 80-bit version has 3 LUTs less and 128-bit version has 4 LUTs less, when implemented in spartan-3 & virtex-4. For need of a better level of security, the key dependent S-box concept is studied. The Key Dependent concept (in [12]) is made practical by introducing a modified version of key dependent architecture is introduced and implemented the same. This architecture boosts the security of PRESENT, as it takes 16 times the period required to break the conventional PRESENT. This concept can be applied to any hardware architecture of PRESENT which largely increases the security level of the cipher with some compromise in the area of architecture. With more advancements to come in the field of cryptography, the area of the architecture can be highly minimized which makes

this concept very handy. The architectures discussed in the paper are simulated using the Xilinx ISE design suite and implemented using various versions of Spartan and Virtex FPGA boards. Thus, the results conclude that the Iterative 4-bit architecture reduces the area where as the Modified Key Dependent architecture increases the security.

REFERENCES

- [1] Z. Zou et al., "A Low-Power and Flexible Energy Detection IR-UWB Receiver for RFID and Wireless Sensor Networks," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 58, no. 7, pp. 1470-1482, July 2011.
- [2] Disruptive Civil Technologies: Six Technologies With Potential Impacts on US Interests Out to 2025, [online] Available: <https://fas.org/irp/nic>.
- [3] D. Giusto, A. Iera, G. Morabito, L. Atzori, The Internet of Things, New York, NY, USA: Springer, pp. 1-32, 2010.
- [4] C. A. Lara-Nino, A. Diaz-Perez and M. Morales-Sandoval, "Lightweight Hardware Architectures for the present Cipher in FPGA," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 64, no. 9, pp. 2544-2555, Sept. 2017.
- [5] C. Alippi, C. Galperti, "An adaptive system for optimal solar energy harvesting in wireless sensor network nodes", IEEE Trans. Circuits Syst. I Reg. Papers, vol. 55, no. 6, pp. 1742-1750, Jul. 2008.
- [6] A. A. R. Haeri, M. G. Karkani, M. Sharifkhani, M. Kamarei, A. Fotowat-Ahmady, "Analysis and design of power harvesting circuits for ultra-low power applications", IEEE Trans. Circuits Syst. I Reg. Papers, vol. 64, no. 2, pp. 471-479, Feb. 2017.
- [7] C. J. Mcivor, M. Mcloone, J. V. McCann, "Hardware elliptic curve cryptographic processor over GF(p)", IEEE Trans. Circuits Syst. I Reg. Papers, vol. 53, no. 9, pp. 1946-1957, Sep. 2006.
- [8] T. Good, M. Benaissa, "Very small FPGA application-specific instruction processor for AES", IEEE Trans. Circuits Syst. I Reg. Papers, vol. 53, no. 7, pp. 1477-1486, Jul. 2006.
- [9] N. Smyth, M. McLoone, J. V. McCann, "WLAN security processor", IEEE Trans. Circuits Syst. I Reg. Papers, vol. 53, no. 7, pp. 1506-1520, Jul. 2006.
- [10] D. Dinu, Y. Le Corre, D. Khovratovich, L. Perrin, J. Großschädl, A. Biryukov, "Triathlon of lightweight block ciphers for the Internet of Things", Proc. NIST Lightweight Cryptogr. Workshop, 2015.
- [11] Information Technology—Security Technologies—Lightweight Cryptography—Part 2: Block Ciphers, Jan. 2012.
- [12] S. S. M. AlDabbagh and I. F. T. A. Shaikhli, "Improving present Lightweight Algorithm," 2013 International Conference on Advanced Computer Science Applications and Technologies, Kuching, 2013, pp. 254-258.
- [13] Rolfes C., Poschmann A., Leander G., Paar C. (2008) Ultra-Lightweight Implementations for Smart Devices – Security for 1000 Gate Equivalents. In: Grimaud G., Standaert FX. (eds) Smart Card Research and Advanced Applications. CARDIS 2008. Lecture Notes in Computer Science, vol 5189. Springer, Berlin, Heidelberg
- [14] N. Hanley M. O'Neill "Hardware comparison of the ISO/IEC 29192-2 block ciphers" Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI) pp. 57-62 Aug. 2012.
- [15] S. R. Prasad, Siripagada, A., Selvaraj, S., and Mohankumar, N., "Random seeding LFSR-based TRNG for hardware security applications", Studies in Computational Intelligence, vol. 771, pp. 427-434, 2019.
- [16] C. S. Pradeep and Ramanathan, R., "Investigation of secret key capacity in MIMO-OFDM wireless systems", in Procedia Computer Science, 2018, vol. 143, pp. 776-785.
- [17] R. Reshma, T. Ramesh and P. Sathishkumar, "Security situational aware intelligent road traffic monitoring using UAVs," 2016 International Conference on VLSI Systems, Architectures, Technology and Applications (VLSI-SATA), Bangalore, 2016, pp. 1-6. doi: 10.1109/VLSI-SATA.2016.7593027